

ASCEND ORGANIZER

RELÁTORIO

RELATÓRIO COMPLETO — ASCEND ORGANIZER

Visão Geral do Projeto

O projeto "Ascend Organizer" foi desenvolvido utilizando Python com o objetivo de automatizar a organização de arquivos da pasta Downloads do usuário.

O sistema identifica automaticamente extensões de arquivos, cria pastas específicas e move os arquivos para seus respectivos diretórios.

Além disso, o projeto possui:

- Interface gráfica moderna
- Sistema de logs
- Arquitetura modular
- Configuração externa via JSON
- Separação de responsabilidades
- Estrutura profissional de software

Objetivo do Projeto

O principal objetivo foi desenvolver um software desktop profissional utilizando Python, aplicando conceitos reais de desenvolvimento de software.

O sistema foi construído para:

- Automatizar tarefas repetitivas
- Melhorar organização de arquivos
- Aplicar arquitetura modular
- Desenvolver habilidades em interface gráfica
- Simular estrutura de software real

Estrutura do Projeto

```
ascend-organizer/  
|  
├── assets/  
├── logs/  
├── config/  
│   └── config.json  
├── core/  
│   ├── organizer.py  
│   └── categories.py
```

```
|   └─ logger.py
|
|─ ui/
|   └─ components.py
|       └─ main_window.py
|
|─ main.py
|─ requirements.txt
└─ README.md
```

Explicação da Estrutura

assets/

Responsável por armazenar arquivos visuais do projeto.

Exemplos:

- ícones
- imagens
- logos

logs/

Armazena os registros do sistema.

Arquivo principal:

`organizer.log`

Responsável por salvar:

- movimentações
- erros
- ações executadas

config/

Armazena configurações externas.

Arquivo:

`config.json`

Permite alterar o comportamento do sistema sem modificar o código Python.

core/

Responsável pela lógica principal do sistema.

Arquivos:

- `organizer.py`
- `categories.py`
- `logger.py`

ui/

Responsável pela interface gráfica.

Arquivos:

- `components.py`
- `main_window.py`

CONFIG.JSON — EXPLICAÇÃO COMPLETA

Código

```
{
  "tema": "dark",
  "monitorar_downloads": true,
  "mostrar_logs": true,
  "pastas": {
    ".png": "Imagens",
    ".jpg": "Imagens",
    ".mp4": "Videos",
    ".pdf": "PDFs"
  }
}
```

Explicação Linha por Linha

"tema": "dark"

Define o tema visual da aplicação.

Possíveis usos:

- dark
- light

"monitorar_downloads": true

Define se o sistema irá monitorar automaticamente a pasta Downloads.

Valor booleano:

- true
- false

"mostrar_logs": true

Controla se os logs serão exibidos no terminal.

"pastas"

Objeto responsável por mapear extensões para categorias.

Exemplo:

```
".png": "Imagens"
```

Significa:

Arquivos `.png` serão movidos para a pasta `Imagens`.

CATEGORIES.PY — EXPLICAÇÃO COMPLETA

Objetivo

Responsável por carregar o arquivo de configuração JSON.

Importações

```
from pathlib import Path  
import json
```

pathlib

Biblioteca moderna para manipulação de caminhos.

json

Biblioteca utilizada para leitura de arquivos JSON.

CONFIG_PATH

```
CONFIG_PATH = Path("config/config.json")
```

Cria o caminho do arquivo de configuração.

carregar_config()

Função responsável por:

- abrir o JSON
- converter JSON em dicionário Python
- retornar os dados

try

Utilizado para tratamento de erros.

Evita que o sistema quebre caso:

- o arquivo não exista
- o JSON esteja inválido

`json.load()`

Converte o conteúdo JSON em dicionário Python.

`obter_categorias()`

Retorna apenas a chave:

`"pastas"`

do arquivo de configuração.

LOGGER.PY — EXPLICAÇÃO COMPLETA

Objetivo

Responsável por registrar logs do sistema.

`datetime`

`from datetime import datetime`

Usado para capturar:

- data atual
- horário atual

LOGS_DIR

```
LOGS_DIR = Path("logs")
```

Define a pasta de logs.

LOG_FILE

```
LOG_FILE = LOGS_DIR / "organizer.log"
```

Define o arquivo de log.

criar_pasta_logs()

Cria automaticamente a pasta logs caso ela não exista.

registrar_log()

Função principal do sistema de logs.

Recebe uma mensagem e:

- adiciona horário

- salva no arquivo
- imprime no terminal

strftime()

Formata a data.

Exemplo:

```
"%d/%m/%Y %H:%M:%S"
```

Resultado:

```
29/05/2026 03:15:20
```

ORGANIZER.PY — EXPLICAÇÃO COMPLETA

Objetivo

Responsável por organizar automaticamente os arquivos.

DOWNLOADS_DIR

```
DOWNLOADS_DIR = Path.home() / "Downloads"
```

Obtém automaticamente a pasta Downloads do usuário.

organizar_arquivos()

Função principal da automação.

obter_categorias()

Lê as categorias definidas no config.json.

for arquivo in DOWNLOADS_DIR.iterdir()

Percorre todos os arquivos da pasta Downloads.

is_file()

Verifica se o item atual é realmente um arquivo.

suffix

Obtém a extensão do arquivo.

Exemplo:

`.png`

`.mp4`

`lower()`

Transforma a extensão em minúsculo para evitar inconsistências.

`if extensao in categorias`

Verifica se a extensão existe no sistema de categorias.

`pasta_destino`

Cria o caminho final da pasta de destino.

`mkdir(exist_ok=True)`

Cria automaticamente a pasta caso ela ainda não exista.

`shutil.move()`

Move o arquivo para a nova pasta.

try / except

Responsável pelo tratamento de erros.

Evita que o programa pare caso aconteça:

- erro de permissão
- arquivo aberto
- conflito de nomes

COMPONENTS.PY — EXPLICAÇÃO COMPLETA

Objetivo

Criar componentes reutilizáveis da interface.

Benefícios

- reutilização
- organização
- padronização visual
- manutenção facilitada

criar_botao()

Cria botões padronizados.

criar_titulo()

Cria títulos padronizados.

criar_card()

Cria containers visuais.

criar_texto()

Cria textos padronizados.

MAIN_WINDOW.PY — EXPLICAÇÃO COMPLETA

Objetivo

Construir toda a interface gráfica.

MainWindow

Classe principal da aplicação.

Herda:

ctk.CTk

init()

Executa automaticamente ao iniciar a janela.

Responsável por:

- configurar janela
- definir tema
- iniciar interface

title()

Define o título da janela.

geometry()

Define tamanho da janela.

set_appearance_mode()

Define o tema visual.

criar_interface()

Função responsável por desenhar todos os elementos da interface.

Sidebar

Área lateral da aplicação.

Contém:

- título
- descrição
- botão principal

Main Area

Área principal do software.

Contém:

- card principal
- mensagens
- informações

MAIN.PY — EXPLICAÇÃO COMPLETA

Objetivo

Ponto de entrada principal do sistema.

MainWindow()

Cria a janela principal do aplicativo.

mainloop()

Mantém a interface aberta.

Responsável por:

- detectar cliques
- atualizar interface
- processar eventos

if name == "main"

Verifica se o arquivo foi executado diretamente.

Boa prática profissional em Python.

FLUXO COMPLETO DO SISTEMA

main.py



main_window.py



components.py



organizer.py
↓
categories.py
↓
config.json
↓
logger.py
↓
organizer.log

CONCEITOS APRENDIDOS

Durante o desenvolvimento foram utilizados conceitos importantes de programação:

- modularização
- orientação a objetos
- tratamento de erros
- manipulação de arquivos
- interface gráfica
- leitura de JSON
- automação
- logs
- arquitetura de software
- componentização

CONCLUSÃO

O projeto Ascend Organizer representa um sistema desktop funcional desenvolvido com Python utilizando arquitetura modular e boas práticas de programação.

O sistema automatiza a organização de arquivos da pasta Downloads e demonstra conhecimentos em:

- automação
- interface gráfica
- organização de software
- manipulação de arquivos
- desenvolvimento desktop